Learning an identity mapping with 3 hidden neurons

Advanced Concepts in Machine Learning Assignment 1

Thai Binh Nguyen, Ege Deniz

1. Network Implementation

The network consists of 8 input neurons, 3 hidden neurons and 8 output neurons. Each input vector contains a 1 and seven 0s, and the target output is equal to the input. We can look at the weights and hidden neuron activations to understand how the inputs are represented.

Both the hidden and output layers have a sigmoid activation function so we can get a continuous output between 0 and 1. All 59 weights and biases are trainable. We have used NumPy for array operations and have implemented forward propagation, backpropagation, and parameter updates manually.

2. Training

The training is done with gradient descent and backpropagation. In the forward pass stage, hidden and output neurons apply the sigmoid to the sum of their weighted inputs and bias. The goal is to minimise the half-squared reconstruction error averaged over eight input vectors:

$$L = \frac{1}{2m} \sum_{i=1}^{m} \| \hat{y}_i - x_i \|^2, \qquad m = 8.$$

Here x_i represents an input vector and $\hat{y}_i$ represents the reconstructed output. The gradients for the weights and biases are calculated by backpropagating the error signal from the output layer to the hidden layer and applying the sigmoid derivative at each layer. The gradient of each weight combines the incoming activation with the neuron's error signal. The bias gradient uses the error signal directly.

All parameters will be updated after calculating the gradients. The gradients are averaged across the 8 inputs then we perform a negative gradient update on all weights and biases. This step uses a learning rate to control how large a step is taken. One epoch is one full batch update. The calculations are vectorised with the inputs stored as rows and biases separate. There is no regularisation or momentum.

The initial weights are sampled from uniform distributions between -0.1s and 0.1s, where s is the scale of the weights. Biases are initialised as zero. Weights are randomised so the hidden neurons can learn different output. We tried multiple learning rates and weight scales.

3. Learning Performance

All gradients were checked using central finite differences. Using the base configuration for the weights with seed 0, inputs 1,3,6, and epsilon 1e-5. The maximum difference was shown as follows:

W1: 1.670e-11, b1: 1.414e-11, W2: 1.253e-11, b2: 4.758e-12 for all parameters. This is less than 1e-7 so the backpropagation gradients closely agree with the numerical estimates at this point. However, this does not prove they are correct for all values.

A run is deemed successful if all outputs are within 0.10 of the target. We check this after each update. If the run is successful or 20,000 epochs have passed then training will stop. We tried 5 learning rates with scale 1 and 3 additional scales with learning rate 3. We ran 80 runs with seeds between 0-9. Each configuration, including the base configuration, used 10 seeds. The base configuration appears in both comparisons, but its 10 runs are counted only once in the total of 80.

Rate	Scale	Success	Median Epochs	Median Seconds	Median Loss
0.1	1	0/10	-	1.033	0.163174
1	1	10/10	10,880.0	0.579	0.006096
3	1	10/10	3,552.5	0.179	0.006057
10	1	10/10	1,054.5	0.051	0.006188
100	1	0/10	-	1.020	0.468749
3	0	0/10	-	0.982	0.016545
3	0.01	10/10	4,557.5	0.223	0.006064
3	5	10/10	3,442.5	0.168	0.006494

Notes: Only successful runs are included in the median epochs. All runs including unsuccessful ones are included in the median time and final loss. Time is measured from the clock on the machine.

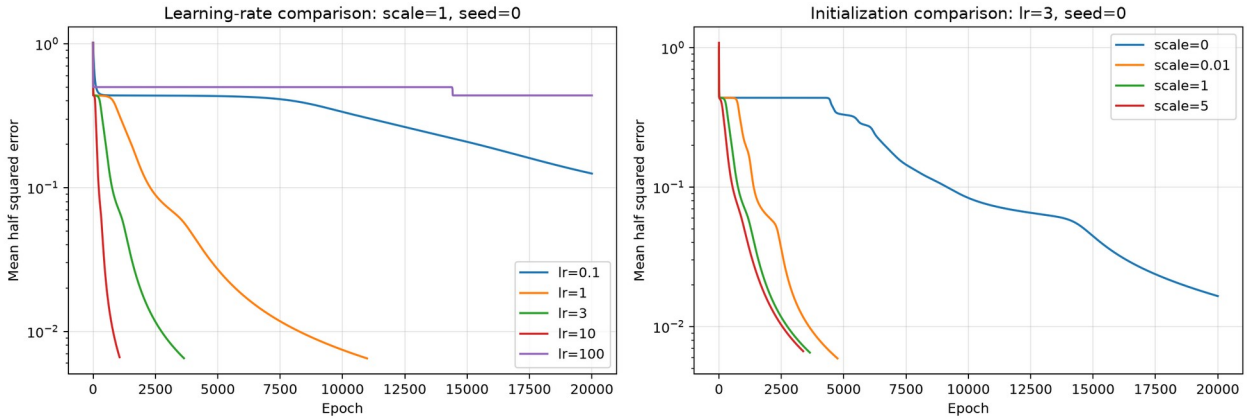


Figure 1 - Learning curves for seed 0. Left: Varying learning rate with scale 1. Right: Varying weight scale with learning rate 3. Training will stop after the network has successfully trained or after the max epoch. Table shows results for all 10 seeds.

The learning rates of 1, 3 and 10 were all successful in all 10 runs. The learning rate of 10 had the fastest median epochs (1054.5) compared to 3 (3552.5) and 1 (10880). The learning rate of 0.1 and 100 did not meet the criteria within the training limit. For the learning rate of 3 all nonzero weight scales were successful (0.01, 1 and 5). The weight scale of 0.01 had the slowest median epochs (4557.5), however the weight scale of 5 was slightly faster than the weight scale of 1. The weight scale of 0 was not successful. This is despite the loss being reduced. The weight scale of 0 causes the same starting parameters to be used for all 10 runs as the seed does not matter at a scale of 0. This shows the model is reliable for these successful settings across the seeds tried, however, this does not guarantee it will work for all initialisations. The model may not converge within the training limit but could still converge later.

4. Weights and Hidden Representation

The base configuration had seed 0, learning rate 3 and weight scale 1. This resulted in a successful run after 3644 epochs (29152 input presentations) in 0.193 seconds. The loss dropped from 1.016208 to 0.006506. The maximum absolute error was 0.099990. All outputs matched the target after thresholding at 0.5.

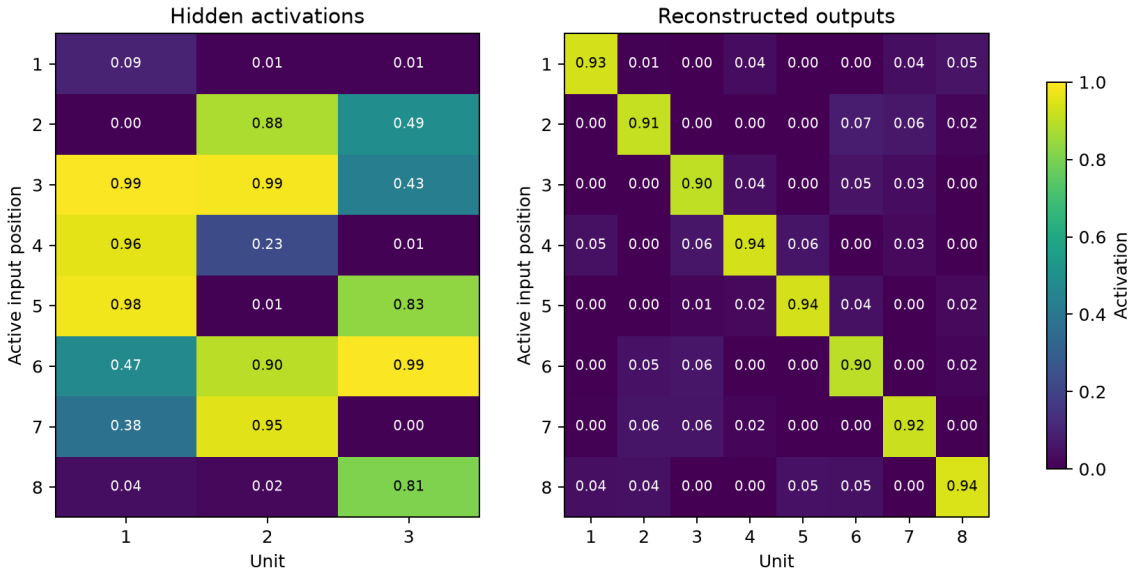


Figure 2 - Hidden activations and reconstructed outputs for the base configuration before thresholding. The ith row shows the hidden activations for the ith input vector (with 1 in position i). Values have been rounded.

Input	H1	H2	H3	Code (Thresh: 0.5)
1	0.094	0.008	0.011	000
2	0.005	0.878	0.491	010
3	0.993	0.986	0.429	110
4	0.959	0.232	0.007	100
5	0.978	0.008	0.834	101
6	0.470	0.898	0.993	011
7	0.378	0.955	0.004	010
8	0.036	0.023	0.807	001

The hidden neurons activate to different subsets of inputs. H1 has a value greater than 0.5 for inputs 3,4,5. H2 activates for inputs 2,3,6,7. H3 activates for inputs 5,6,8. Inputs 2 and 7 have the same code after thresholding but their activations are different. Particularly for H3 they have very different values (0.491 and 0.004). The network has learnt 8 unique continuous representations but only 7 thresholded codes. The output layer uses the continuous values to differentiate between the inputs.

The learnt weights help to explain why this happened. The weight from input 1 to H1 is -2.320 with a bias of 0.054. So the pre-activation is -2.266 and the activation is ~0.094. The weights to output 7 are [-2.132, 6.567, -11.176] for H1, H2, H3 with a bias of -2.978. The weight from H3 to output 7 is negative so output 7 will have a higher suppression for input 2. This is because input 7 has a much lower H3 activation. All 3 activations and the bias affect what the output will be. This is for the 8 training inputs and shows the model will not necessarily work on unseen combinations and noisy inputs.

5. Software and References

Install NumPy, Matplotlib and Jupyter with command: python -m pip install numpy matplotlib jupyterlab. Open lab1.ipynb with Lab1 as the working directory. Restart the kernel and run all cells. This will create the inputs, run the 80 trials and save the tables, figures and baseline_model.npz file to results/. If you run it again, it will overwrite these files.

Lab - NN.pdf
ACML Assignment 1 specification.

Artificial Neural Networks (1.2-ANN.pdf)
ACML Course Material.

Backprop Help.pdf
Additional notes on backpropagation and gradient checking.